PRACTICAL 13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

AIM

To build PHP JSON API endpoints for campus events and consume them using JavaScript Fetch API for asynchronous list, add, update and delete operations.

Introduction

The new API reads event records from MySQL and returns JSON with HTTP status codes. Public requests can list or read events. Create, update and delete operations require an administrator session and CSRF token. An admin-only demo page uses Fetch and updates the list without a full page reload.

Tools and Technology

VS Code, XAMPP, PHP, MySQLi, MariaDB, HTML, CSS, JavaScript Fetch API, browser, Git and GitHub.

Files Used

File	Purpose
php/api-events.php	JSON GET, POST, PUT and DELETE API with validation and role checks.
api-events-demo.php	Admin-only Fetch client and event form.
js/api-demo.js	Asynchronous list, create, edit and delete requests.
php/event-functions.php	Shared event validation, prepared statements and audit helper.
docs/Practical13-setup.md	Local setup and browser Network panel steps.
scripts/test-practical13.py	Repeatable authenticated API integration checks.
scripts/test-practical13-input.php	Database-free validation checks.

Key Skills Used

PHP JSON API, HTTP methods and status codes, Fetch API, asynchronous CRUD, JSON parsing, server-side validation, CSRF protection and frontend/backend integration.

Implementation Overview

1 JSON Read Endpoints

GET returns an event collection with a count or one event by ID. Event fields are selected from the MySQL database using prepared statements. Missing and malformed identifiers return explicit JSON errors.

2 Create and Update

POST creates an event and returns HTTP 201. PUT updates an event by ID. Both methods validate title, date, venue, category and status before database writes, then record the admin action.

3 Delete and Referential Checks

DELETE removes only an event that has no registrations. A referenced event returns HTTP 409 and remains available for closure through the event management page.

4 Access and Error Handling

Read requests are public. Mutations require a current admin role and the session CSRF token in X-CSRF-Token. Validation failures use HTTP 422, unauthorized requests use 401 or 403, and unexpected failures return a generic 500 response.

5 Fetch Client without Reload

The admin demo loads JSON on page open, submits JSON with Fetch and refreshes the table after each successful change. Errors appear in a live status message. Event text is rendered with textContent to avoid treating API data as HTML.

Implementation Evidence

The live API passed 9 authenticated integration checks, including JSON reads, anonymous access denial, validation, create, fetch-by-ID, update, delete and missing-record handling. Live browser screenshots remain to be captured.

Database Results

Check	Observed result
API data source	MySQL events table through prepared queries.
Mutation audit	Practical 14 audit_logs and the existing event_audit table.
Credential handling	No API password, cookie or token is returned in JSON.

Testing Summary

Test input or action	Observed result
Database-free event validation	4 checks passed: valid data, date, category/status and length limits.
Practical 13 API integration suite	9 checks passed: JSON reads, access denial, validation, create, fetch, update, delete and missing record.

Screenshots of Portal Implementation

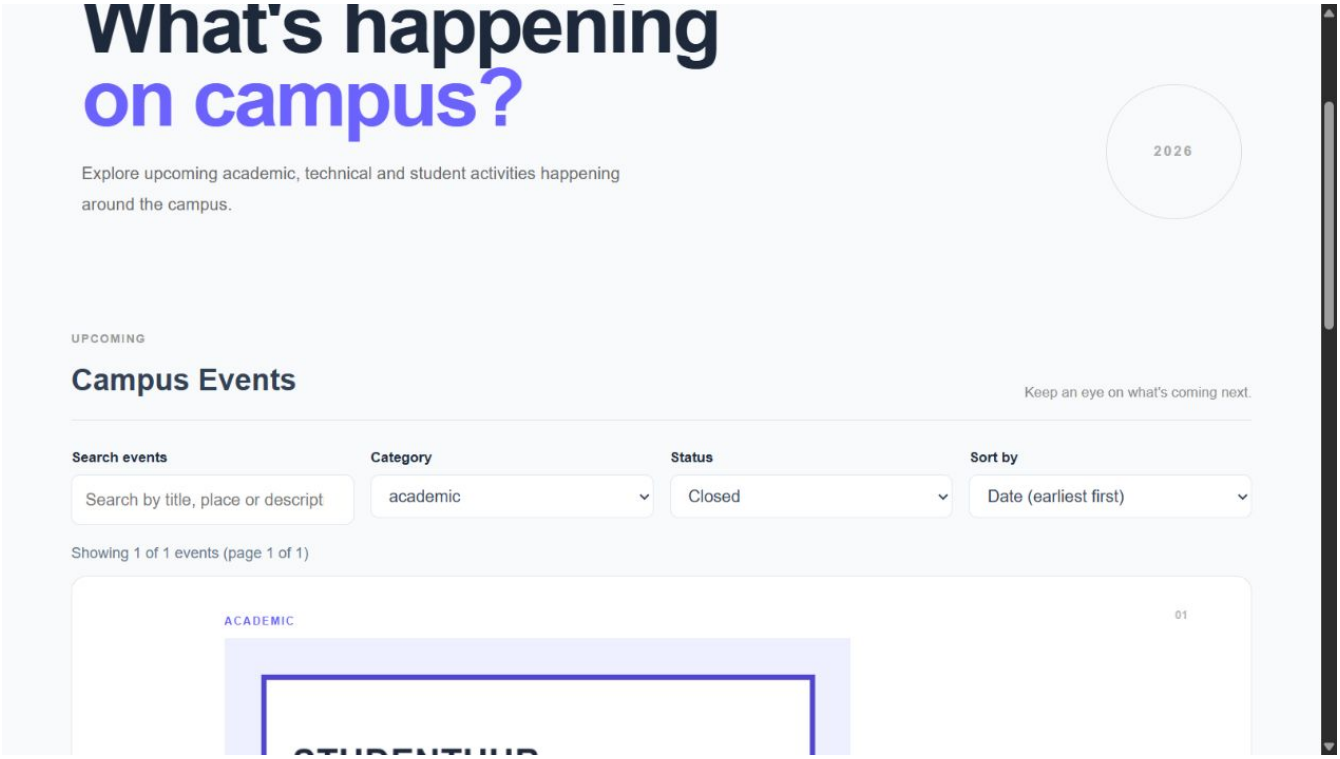


Figure 1: Existing campus events page, showing the public event listing used by the portal.

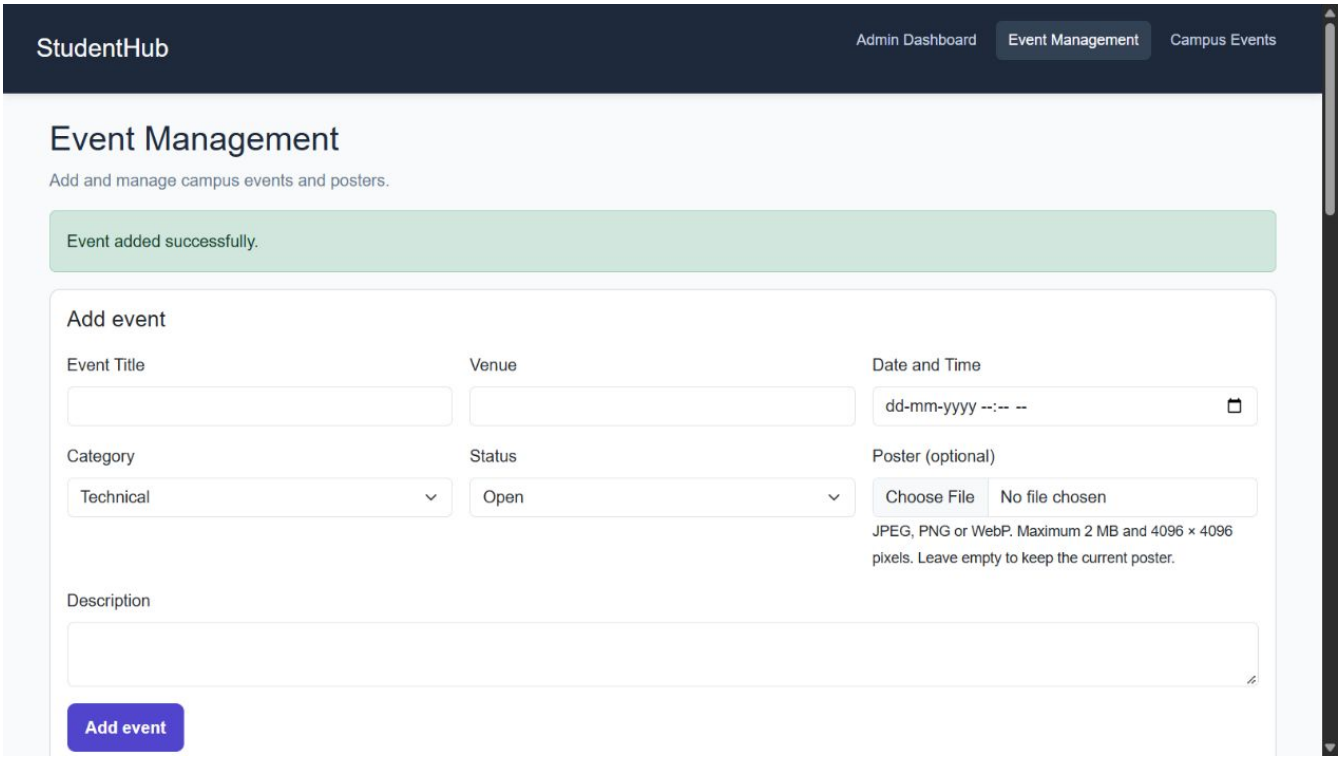


Figure 2: Existing event-management success state from Practical 12; this is supporting UI context, not a Fetch API screenshot.

Key Questions and Answers

Q1 Do PHP endpoints return valid JSON responses

The endpoint returns JSON for successful reads and mutations, and JSON error objects for rejected requests. The runtime checks are provided but need the configured local XAMPP database.

Q2 Are GET and POST requests handled properly

GET supports the event collection and an individual ID. POST creates an event. PUT updates and DELETE removes an eligible event. Unsupported methods return HTTP 405.

Q3 Are loading, success and error states shown on the frontend

Yes. The Fetch demo displays loading text, success messages and API errors through a role=status live region.

Q4 Is input validation performed before database operations

Yes. The existing shared event validator checks lengths, date, category and status before prepared INSERT or UPDATE statements execute.

GitHub Repository

https://github.com/DarshilRupani/studenthub-portal
Practical 13–15 files and reports are committed and pushed to the main branch.

Conclusion

The live API and authenticated CRUD workflow passed all 9 integration checks. Screenshots from the running API demo and Network panel remain to be added if required by the instructor.